Guia operativa: fine-tuning exitoso de LLMs pequenos y medianos en hardware limitado

Fecha de registro: 2026-05-09

1. Proposito

Esta guia documenta el procedimiento operativo para entrenar, adaptar y evaluar modelos de lenguaje pequenos o medianos en hardware no especializado, usando tecnicas de cuantizacion y LoRA/QLoRA. Esta pensada para una compania de software que quiere crear modelos locales con mejor disciplina de trabajo en repositorios: respetar alcance, evitar cambios fuera de scope, validar honestamente, no inventar resultados y reportar riesgos.

El objetivo no es competir con un frontier model generalista. El objetivo es construir un worker local que haga bien tareas concretas de ingenieria dentro de nuestros repositorios y que pueda integrarse con flujos tipo Codex/OpenCode.

2. Principio central

El fine-tuning pequeno funciona mejor cuando se entrena comportamiento operativo, no conocimiento general. No intentamos ensenarle al modelo "todo Django", "todo React" o "toda seguridad". Le damos ejemplos repetidos y limpios de como debe actuar:

- leer el contexto antes de editar;
- obedecer archivos permitidos y prohibidos;
- reutilizar patrones existentes;
- preferir cambios pequenos y verificables;
- reportar bloqueos en vez de inventar;
- no tocar secretos, datos privados, caches, binarios ni artefactos generados;
- escribir resultados factuales.

En hardware limitado, la calidad del dataset importa mas que el tamano del entrenamiento.

3. Hardware de referencia validado

Configuracion validada durante esta sesion:

- Windows host con WSL2 Ubuntu.
- GPU NVIDIA RTX 5070 Laptop.
- VRAM real detectada por Unsloth: 7.96 GB.
- RAM del sistema: 32 GB.
- CPU Ryzen 9 HX.
- Python virtualenv en WSL.
- CUDA visible desde PyTorch.
- Modelo base: `unsloth/Qwen3-14B-unsloth-bnb-4bit`.

Resultado validado:

- Qwen3 14B cargo en 4-bit.
- Inferencia local funciono.
- LoRA se adjunto correctamente.
- Entrenamiento smoke de 1 step funciono.

- Adapter se guardo y recargo.
- Entrenamiento de 5 steps funciono.

Esto es una prueba fuerte de viabilidad, pero no significa que un entrenamiento largo de 14B sea comodo en 8 GB de VRAM. Para produccion local sostenida, 7B suele ser mas practico; 14B puede usarse para pruebas controladas y corridas pequenas.

4. Arquitectura recomendada

Usar una separacion clara entre repos de producto y workspace de entrenamiento:

```
Repos de producto:
  C:\Users\&lt;user&gt;\Programacion\Programacion\&lt;repo&gt;

Workspace de preparacion en Windows:
  C:\Users\&lt;user&gt;\ai-finetune-qwen

Workspace de entrenamiento en WSL:
  ~/ai-finetune-qwen
```

El workspace Windows se usa para cosechar y preparar datasets desde repos locales. El workspace WSL se usa para CUDA, Unsloth, PyTorch y entrenamiento. Esta separacion reduce conflictos entre herramientas Windows y Linux.

5. Seleccion de modelo

Regla practica:

- 1B a 3B: bueno para validar pipeline, barato, rapido, calidad limitada.
- 7B: mejor balance para laptops y GPUs de 8 GB a 12 GB.
- 14B: viable con 4-bit y LoRA muy conservador, lento y al limite.
- 30B o mas: usar cloud o GPU con mas VRAM.

Para hardware limitado, empezar siempre con un modelo cuantizado 4-bit compatible con el stack de entrenamiento. En este caso, el modelo `unsloth/Qwen3-14B-unsloth-bnb-4bit` ya trae configuracion preparada para Unsloth/bitsandbytes.

6. Preparacion del entorno

El entorno debe validarse por capas. No entrenar hasta que cada capa pase.

6.1 CUDA

Confirmar que PyTorch ve la GPU:

```
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available());
    print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'no cuda')"
```

Exito esperado:

```
True
NVIDIA ...
```

6.2 Librerias

Confirmar import del stack:

```
python -c "import torch, transformers, datasets, accelerate, peft, trl; print('training stack
    ok')"
```

Para Unsloth:

```
python -c "import unsloth; print(unsloth.__version__)"
```

Importante: en scripts reales, importar Unsloth antes que `transformers`, `trl` y `peft`.

7. Dataset: que entrenar

El dataset debe ensenar el comportamiento que queremos repetir. Recomendamos tres tipos de ejemplos:

7.1 SFT positivo

Ejemplos donde el worker recibio una tarea, respeto scope, edito archivos correctos y valido.

Formato conceptual:

```
{
  "messages": [
    {"role": "system", "content": "Reglas del worker"},
    {"role": "user", "content": "Tarea, allowed files, forbidden files, contexto, validacion"},
    {"role": "assistant", "content": "Solucion esperada, resumen, patch o resultado factual"}
  ]
}
```

7.2 Rejection cases

Ejemplos donde el comportamiento correcto es no proceder:

- tarea pide secretos;
- scope no permite editar;
- cambio destructivo no autorizado;
- falta contexto critico;
- riesgo de tenant/customer data;
- migracion o dependencia no autorizada.

Estos ejemplos son cruciales para que el modelo aprenda a detenerse.

7.3 Eval held-out

Prompts que nunca se entrenan. Se usan para comparar modelo base vs adapter. Si se entrena sobre los evals, la evaluacion deja de medir generalizacion.

8. Cosecha de datos desde proyectos terminados

Una forma poderosa de generar dataset es convertir commits terminados en ejemplos de entrenamiento. Un commit bueno tiene:

- una tarea implicita en el mensaje de commit;
- archivos tocados reales;
- diff de oro;
- contexto del repo;
- validacion probable;
- evidencia de que el cambio ya fue aceptado.

Procedimiento recomendado:

1. Seleccionar repos terminados o en estado estable. 2. Extraer commits recientes. 3. Excluir commits demasiado grandes. 4. Excluir binarios, caches, bases de datos, imagenes y builds. 5. Escanear secretos y credenciales. 6. Convertir cada commit en chat JSONL. 7. Mantener el harvest en archivo separado antes de mezclar.

Ejemplo de fila generada:

```
SYSTEM:
You are a disciplined local coding worker...

USER:
Project: noah_app
Commit: 111978510062
Task: feat: tighten dispatch handoff across kitchen and driver ops
Allowed files: backend/core/drivers/selectors.py, ...
Forbidden files: unrelated files, secrets, generated artifacts, dependency/vendor caches
Expected validation: run focused Python tests if present; run focused frontend tests or build if
     present

ASSISTANT:
Implement the completed change...
Keep edits scoped to...
```

```
Validation expectation...
Gold patch excerpt:
```

+def _driver_visible_delivery_q(driver): + ...

El modelo no recibe "tokens secretos". Recibe texto tokenizado de entrenamiento. Debemos impedir que ese texto incluya API keys, passwords, tokens de auth, `.env`, dumps, datos de clientes o URLs privadas.

9. Scrubbing obligatorio

Antes de entrenar, aplicar filtros conservadores:

- `api_key`, `secret`, `token`, `password`, `passwd`, `private_key`;
- `sk_live_`, `pk_live_`;
- `django-insecure`;
- `SECRET_KEY`;
- `.env`, `.env.*`;
- bases de datos: `.db`, `.sqlite`, `.sqlite3`;
- binarios, imagenes, PDFs, zips;
- caches: `__pycache__`, `.pytest_cache`, `node_modules`, `dist`, `build`;
- browser profiles y artefactos de herramientas.

Si hay duda, excluir. Es mejor perder 10 ejemplos que entrenar secretos o basura.

10. Escalera de validacion

Nunca empezar con entrenamiento largo. Usar esta escalera:

Fase 1: validar JSONL

Confirmar que cada fila tiene `system`, `user`, `assistant`.

Fase 2: validar tokenizer

Aplicar chat template a una fila y revisar el texto renderizado.

Fase 3: cargar modelo

Cargar modelo cuantizado en 4-bit sin entrenar.

Fase 4: inferencia minima

Pedir una respuesta fija:

```
Say only: ready /no_think
```

Para Qwen3, usar `enable_thinking=False` en el chat template y agregar `/no_think` cuando convenga.

Fase 5: adjuntar LoRA

Probar `FastLanguageModel.get_peft_model` sin entrenar. Confirmar parametros entrenables.

Fase 6: smoke train

Entrenar 1 step, batch 1, secuencia corta. Guardar nada o guardar adapter de prueba.

Fase 7: guardar y recargar adapter

El ciclo minimo exitoso es:

```
load base -&gt; attach LoRA -&gt; train 1 step -&gt; save adapter -&gt; reload adapter -&gt; inference
```

Fase 8: entrenamiento pequeno

Subir a 5 o 10 steps. Solo despues considerar mas pasos.

11. Parametros conservadores para 8 GB VRAM

Configuracion que funciono para Qwen3 14B en RTX 5070 Laptop:

```
max_seq_length = 128
r = 4
target_modules = ["q_proj", "k_proj", "v_proj", "o_proj"]
lora_alpha = 8
per_device_train_batch_size = 1
gradient_accumulation_steps = 1
optim = "adamw_8bit"
bf16 = True
fp16 = False
packing = False
```

Variable critica para evitar fallo de fused cross entropy:

```
UNSLOTH_CE_LOSS_N_CHUNKS=256
```

Sin esa variable, Unsloth puede fallar con:

```
No or negligible GPU memory available for fused cross entropy.
```

12. Manejo de memoria en WSL

Despues de cargar modelos grandes, WSL puede retener RAM. Para limpiar:

En PowerShell de Windows:

```
wsl --shutdown
```

Luego abrir WSL y reactivar entorno:

```
cd ~/ai-finetune-qwen &amp;&amp; source .venv/bin/activate
```

Usar este paso antes de entrenamientos nuevos si se acaba de cargar o entrenar un modelo grande.

13. Problemas observados y soluciones

13.1 Descarga aparentemente atascada

Si Hugging Face queda en un porcentaje fijo, cancelar con `Ctrl+C` y reintentar con `snapshot_download`. La cache suele reanudar.

13.2 Offload 4-bit incompatible

En el stack probado, offload CPU con `transformers/accelerate/bitsandbytes` fallo con:

```
Params4bit.__new__() got an unexpected keyword argument '_is_hf_initialized'
```

La solucion practica fue no depender de offload para entrenamiento y cargar directamente en GPU con 4-bit, manteniendo secuencia y LoRA muy pequenos.

13.3 Falta `Python.h`

Triton fallo compilando si faltaban headers:

```
fatal error: Python.h: No such file or directory
```

Solucion en WSL:

```
sudo apt-get update &amp;&amp; sudo apt-get install -y python3.12-dev
```

13.4 Qwen responde con `<think>`

Usar:

```
tokenizer.apply_chat_template(..., enable_thinking=False)
```

Y en prompts cortos:

```
/no_think
```

14. Interpretacion de resultados

No obsesionarse con loss en corridas de 1 a 10 steps. En esa fase se valida infraestructura:

- el modelo carga;
- la loss calcula;
- backward funciona;
- grad_norm existe;
- adapter se guarda;
- adapter se recarga;
- inferencia posterior funciona.

En la prueba validada:

- 1 step funciono con loss aproximado 4.808.
- 5 steps funcionaron con loss promedio aproximado 4.634.
- Adapter r=4 atencion-only se guardo correctamente.

La mejora real debe medirse con evals held-out, no con sensacion subjetiva.

15. Evaluacion

Mantener una carpeta de eval prompts no entrenados. Cada eval debe especificar:

- tarea;
- archivos permitidos;
- archivos prohibidos;
- contexto quirurgico;
- criterios de exito;
- comandos de validacion;
- fallos a evitar.

Comparar:

1. Modelo base. 2. Modelo base + adapter smoke. 3. Modelo base + adapter entrenado.

La evaluacion debe revisar:

- obediencia de scope;
- factualidad del reporte;
- si inventa rutas o APIs;
- si toca archivos no permitidos;
- si evita secretos;
- si valida honestamente.

16. Politica de versionado de adapters

Cada adapter debe tener nombre descriptivo:

```
qwen14b-codex109-attn-r4-steps10
```

El nombre debe codificar:

- modelo base;

- dataset o numero de filas;
- tipo de LoRA;
- rank;
- steps;
- fecha si hay varias corridas.

Guardar junto al adapter:

- dataset usado;
- commit o hash del dataset;
- parametros de entrenamiento;
- logs de loss;
- resultados de eval;
- notas de hardware.

17. Recomendaciones para hardware limitado

Para una laptop de 8 GB VRAM:

- priorizar 7B para entrenamiento util local;
- usar 14B para pruebas pequenas o adapters muy conservadores;
- mantener `max_length` bajo al principio;
- usar `r=4` o `r=8`;
- empezar con atencion-only;
- usar dataset pequeno pero limpio;
- aumentar steps antes que aumentar contexto;
- limpiar WSL entre corridas;
- no usar MLP LoRA hasta que haya margen;
- no mezclar datasets sin validacion.

18. Criterios para escalar

Escalar solo si se cumplen todos:

- JSONL validado sin errores;
- secret scan limpio;
- tokenizer renderiza bien;
- modelo carga;
- inferencia responde;
- LoRA se adjunta;
- 1 step entrena;
- adapter se guarda;
- adapter se recarga;
- eval minimo no empeora comportamiento.

Escalamiento recomendado:

```
1 step -&gt; 5 steps -&gt; 10 steps -&gt; 25 steps -&gt; 1 epoch parcial -&gt; 1 epoch
```

En cada salto, registrar tiempo, loss, memoria maxima y calidad de eval.

19. Lo que no se debe hacer

- Entrenar directo con todo el disco.
- Entrenar con `.env`, bases de datos, capturas, caches o browser profiles.
- Mezclar evals en train.
- Aumentar contexto y LoRA rank al mismo tiempo.
- Confiar en que un commit grande es un buen ejemplo.
- Entrenar secretos pensando que "es local".
- Medir exito solo por loss.
- Saltar el paso de recargar adapter.

20. Runbook minimo

Preparar

```
cd ~/ai-finetune-qwen
source .venv/bin/activate
```

Validar dataset

```
python - &lt;&lt;'PY'
import json
from pathlib import Path

p = Path("datasets/final-train.with-codex-projects.chat.jsonl")
rows = 0
bad = 0
for i, line in enumerate(p.open(encoding="utf-8"), 1):
    try:
        row = json.loads(line)
        assert [m["role"] for m in row["messages"]] == ["system", "user", "assistant"]
        rows += 1
    except Exception as e:
        bad += 1
        print("BAD", i, type(e).__name__, e)
print("rows:", rows)
print("bad:", bad)
PY
```

Entrenar de forma conservadora

Usar:

```
UNSLOTH_CE_LOSS_N_CHUNKS=256
```

Con:

```
max_length=128
batch=1
r=4
target_modules=q_proj,k_proj,v_proj,o_proj
max_steps=10
```

Limpiar despues

En PowerShell:

```
wsl --shutdown
```

21. Conclusion

Entrenar un LLM pequeno o mediano en hardware limitado es viable si se trata como ingenieria de sistemas, no como magia. La ruta exitosa es incremental: validar entorno, validar dataset, cargar modelo, inferir, adjuntar LoRA, hacer smoke train, guardar adapter, recargar, evaluar y recien entonces escalar.

La ventaja competitiva no esta solo en entrenar el modelo. Esta en capturar trabajo real de la compania, limpiarlo, convertirlo en ejemplos operativos de alta calidad y usarlo para ensenar disciplina de worker local. Esa memoria procedural es dificil de comprar y muy valiosa cuando se construye con cuidado.